

Code-along 02

FirstName LastName

Setup

Install new package

This chunk will not run (on purpose!) when rendered because `eval: false`.

Install the [summarytools](#) package, available on CRAN. Click the green arrow icon to run the following code chunk.

```
install.packages("summarytools")
```

Load the packages

Load the standard packages and our new package `summarytools()`.

```
library(tidyverse)
library(haven) # not core tidyverse
library(gssr)
library(gssrdoc)
library(summarytools)
```

Load your data

```
# Get the data only for the 2024 survey respondents
gss24 <- gss_get_yr(2024)

# Get the data all survey years
data(gss_all)
```

Use the `gss_which_years()` function to identify the survey years a variable appeared.

```
gss_which_years(gss_all, meovrwrk)
```

```
# A tibble: 35 x 2
  year      meovrwrk
  <dbl> <lgl>
1 1972      FALSE
2 1973      FALSE
3 1974      FALSE
4 1975      FALSE
5 1976      FALSE
6 1977      FALSE
7 1978      FALSE
8 1980      FALSE
9 1982      FALSE
10 1983      FALSE
# i 25 more rows
```

Which survey years did the variable `childs` appear?

```
# REPLACE THE BLANK LINE WITH THE NAME OF THE VARIABLE
gss_which_years(gss_all, _____)
```

Variable Types

Data types in R

A property is assigned to objects that determines how generic functions operate with it.

logical - Boolean values `TRUE` and `FALSE`

```
class(TRUE)
```

```
[1] "logical"
```

character - character strings

```
class("Sociology")
```

```
[1] "character"
```

Integer - numeric data without decimals
(indicated with an L).

```
class(2L)
```

```
[1] "integer"
```

numeric - default type if values are numbers or if the values contain decimals.

```
class(2.5)
```

```
[1] "numeric"
```

factors consist of character data with a fixed and known set of possible values

```
opinion <- factor(c("like", "dislike", "dislike", "hate", "dislike", "hate"))  
class(opinion)
```

```
[1] "factor"
```

```
# By default, the levels are sorted alphabetically.  
levels(opinion)
```

```
[1] "dislike" "hate"    "like"
```

```
# Reorder the levels with the argument `levels` in the `factor()` function  
opinion <- factor(opinion, levels = c("hate", "dislike", "like"))  
levels(opinion)
```

```
[1] "hate"    "dislike" "like"
```

```
# If the order has meaning (like rankings), you can make it an ordered factor
opinion <- factor(opinion, levels = c("hate", "dislike", "like"), ordered = TRUE)
levels(opinion)
```

```
[1] "hate"      "dislike" "like"
```

Converting between types

Use a function: `as.logical()`, `as.numeric()`, `as.integer()`, or `as.character()`.

Create a numeric variable.

```
x <- 1:3
x
```

```
[1] 1 2 3
```

```
class(x)
```

```
[1] "integer"
```

Change it to a character variable.

```
y <- as.character(x)
y
```

```
[1] "1" "2" "3"
```

```
class(y)
```

```
[1] "character"
```

Haven labelled

When you import data into R from software like SPSS, Stata, or SAS, you might notice a special class called `haven_labelled`.

```
class(gss24$premarsx)
```

```
[1] "haven_labelled" "vctrs_vctr"      "double"
```

```
table(gss24$premarsx)
```

```
  1    2    3    4  
438 172 317 1633
```

It makes data easier to understand without needing a separate codebook.

```
attr(gss24$premarsx, "label")
```

```
[1] "Sex before marriage"
```

See the value labels with `print_labels()`

```
print_labels(gss24$premarsx)
```

Labels:

value	label
1	always wrong
2	almost always wrong
3	wrong only sometimes
4	not wrong at all
5	other
NA(d)	don't know
NA(i)	iap
NA(j)	I don't have a job
NA(m)	dk, na, iap
NA(n)	no answer
NA(p)	not imputable
NA(r)	refused
NA(s)	skipped on web
NA(u)	uncodeable
NA(x)	not available in this release
NA(y)	not available in this year
NA(z)	see codebook

You can use `as_factor` to see the value labels of the variable `premarsx`.

```
# REPLACE THE BLANK LINE WITH THE NAME OF THE VARIABLE  
  
table(as_factor(gss24$_____), useNA = "ifany")
```

Convert labels to factors

1. Get rid of all the ‘missing’ (NA) levels using `zap_missing`

```
# REPLACE THE BLANK LINES WITH THE NAME OF THE VARIABLE  
  
gss24$premarsx <- zap_missing(gss24$_____)  
  
table(as_factor(gss24$_____), useNA = "ifany") # see the results
```

2. Apply the labels instead of numeric values using `as_factor`

```
# REPLACE THE BLANK LINE WITH THE FUNCTION `as_factor()`  
  
gss24$premarsx <- _____(gss24$premarsx) # replaces the values with labels  
table(gss24$premarsx, useNA = "ifany") # no longer need to wrap the variable in as_factor
```

3. Get rid of the empty levels in `premarsx` using `droplevels`

```
# REPLACE THE BLANK LINES WITH THE `droplevels()` function & the NAME OF THE VARIABLE  
  
gss24$_____ <- _____(gss24$_____)  
table(gss24$_____)
```

Now, use `zap_missing()`, `as_factor()`, and `droplevels()` to do the same for the `sex` variable.

Then, use `table()` to see your results.

```
# ADD YOUR CODE HERE
```

Frequency Distributions

Make a frequency table of the variable `sex`.

```
freq(gss24$sex)
```

Tagged NA values were detected and will be reported as regular NA; use `haven::as_factor()` to

Frequencies

`gss24$sex`

Label: Respondents sex

Type: Numeric (labelled)

	Freq	% Valid	% Valid Cum.	% Total	% Total Cum.
male [1]	1787	45.17	45.17	44.83	44.83
female [2]	2169	54.83	100.00	54.42	99.25
<NA>	30			0.75	100.00
Total	3986	100.00	100.00	100.00	100.00

Make a pretty frequency table for the variable `premarsx`.

```
# ADD YOUR CODE HERE
```

Using `report.nas = FALSE` suppresses the missing data.

The `headings = FALSE` parameter suppresses the heading section. Do the same for `premarsx`.

```
freq(gss24$sex, report.nas = FALSE, headings = FALSE)
```

Tagged NA values were detected and will be reported as regular NA; use `haven::as_factor()` to

	Freq	%	% Cum.
male [1]	1787	45.17	45.17
female [2]	2169	54.83	100.00
Total	3956	100.00	100.00

```
# ADD YOUR CODE HERE
```

Use these options with the `freq()` function for the variable `premarsx`.

```
# ADD YOUR CODE HERE
```

Cross-tabs

We've been using the `table()` function with one variable at a time, but it also lets you create a frequency table (**`crosstab`**) with two variables.

```
# 1st variable is the rows, 2nd variable is the columns.  
table(gss24$premarsx, gss24$sex)
```

```
      1   2  
1 180 256  
2  71  98  
3 155 161  
4 734 891
```

To run `freq()` by group, pair it with the `stby()` function.

```
stby(gss24$premarsx, gss24$sex, freq)
```

NA detected in grouping variable(s); consider using `useNA = TRUE`

Tagged NA values were detected and will be reported as regular NA; use `haven::as_factor()` to
Tagged NA values were detected and will be reported as regular NA; use `haven::as_factor()` to

Frequencies

`gss24$premarsx`

Label: Sex before marriage

Type: Numeric (labelled)

Group: sex = 1

	Freq	% Valid	% Valid Cum.	% Total	% Total Cum.
always wrong [1]	180	15.79	15.79	10.07	10.07

almost always wrong [2]	71	6.23	22.02	3.97	14.05
wrong only sometimes [3]	155	13.60	35.61	8.67	22.72
not wrong at all [4]	734	64.39	100.00	41.07	63.79
other [5]	0	0.00	100.00	0.00	63.79
<NA>	647			36.21	100.00
Total	1787	100.00	100.00	100.00	100.00

Group: sex = 2

	Freq	% Valid	% Valid Cum.	% Total	% Total Cum.
always wrong [1]	256	18.21	18.21	11.80	11.80
almost always wrong [2]	98	6.97	25.18	4.52	16.32
wrong only sometimes [3]	161	11.45	36.63	7.42	23.74
not wrong at all [4]	891	63.37	100.00	41.08	64.82
other [5]	0	0.00	100.00	0.00	64.82
<NA>	763			35.18	100.00
Total	2169	100.00	100.00	100.00	100.00

Use `summarytools::ctable` instead!

```
ctable(gss24$premarsx, gss24$sex,
       prop = "c",
       format = "p",
       useNA = "no")
```

Cross-Tabulation, Column Proportions

premarsx * sex

Data Frame: gss24

	sex	1	2	Total
premarsx				
1	180 (15.8%)	256 (18.2%)	436 (17.1%)	
2	71 (6.2%)	98 (7.0%)	169 (6.6%)	
3	155 (13.6%)	161 (11.5%)	316 (12.4%)	
4	734 (64.4%)	891 (63.4%)	1625 (63.8%)	
Total	1140 (100.0%)	1406 (100.0%)	2546 (100.0%)	

Show your knowledge

Based on your table:

- *[your answer here]* percentage of respondents believe sex before marriage is ‘almost always wrong’?
- A greater percentage of *[men or women]* think sex before marriage is ‘not wrong at all’.

Data Management I

The pipe |>

The pipe operator passes what comes before it into the function that comes after it as the first argument in that function.

```
sum(1, 2)
```

```
[1] 3
```

```
1 |>  
  sum(2)
```

```
[1] 3
```

dplyr() in action

Create a frequency table for `childs` variable.

```
gss_all$childs <- zap_missing(gss_all$childs)  
gss_all$childs <- as_factor(gss_all$childs)  
gss_all$childs <- droplevels(gss_all$childs)  
  
gss_all |>  
  freq(childs, report.nas = FALSE) |>  
  tb()
```

```
# A tibble: 9 x 4
  childs    freq    pct pct_cum
  <fct>    <dbl>  <dbl>  <dbl>
1 0      20956 27.8    27.8
2 1      11979 15.9    43.7
3 2      18989 25.2    68.9
4 3      11671 15.5    84.3
5 4       5996  7.95   92.3
6 5       2669  3.54   95.8
7 6       1381  1.83   97.7
8 7        734  0.973  98.6
9 8 or more 1032  1.37  100
```

Use a `|>` & `gss_which_years()` to find out which survey years the variable `agekdbrn` appears.

```
# ADD YOUR CODE HERE
```

`filter()` & `drop_na()`

Use `filter()` to keep rows that meet a condition.
Use `drop_na()` to remove rows with missing (NA) values.

```
gss_all |>
  filter(year == 2022) |>
  drop_na(sex, agekdbrn) |>
  count(sex)
```

```
# A tibble: 2 x 2
  sex      n
  <dbl+lbl> <int>
1 1 [male]  1031
2 2 [female] 1363
```

Limit the dataframe to only the 2024 respondents without missing data for `sex`, `hrs1`, and `childs`. How many men and women are left?

```
gss_all |>
  filter(year == 2022) # CHANGE THE YEAR TO 2024 & ADD A PIPE OPERATOR
  drop_na() |> # ADD THE VARIABLES INSIDE drop_na()
  count(sex)
```

Summary Statistics

Median & Mean w base R

```
# Using base R
median(gss_all$agekdbnr, na.rm=TRUE)
```

```
[1] 23
```

Note: `na.rm` is a logical evaluating to `TRUE` or `FALSE` indicating whether NA values should be stripped before the computation proceeds.

```
mean(gss_all$agekdbnr, na.rm=TRUE)
```

```
[1] 24.14491
```

```
# show me summary statistics
summary(gss_all$agekdbnr)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NAs
9.00	20.00	23.00	24.14	28.00	97.00	46832

Find the mean and median for the variable: `hrs1`.

```
# REPLACE THE BLANK LINES WITH THE DATA FRAME NAME & VARIABLE NAME
median(_____, na.rm=TRUE)
mean(_____, na.rm=TRUE)

# USE summary() TO SHOW SUMMARY STATISTICS
```

`summarise()`

Use functions inside `summarise()` to create summary variables.

```
gss_all |>
  drop_na(agekdbnr) |>
  summarise(
    avg = mean(agekdbnr)
  )
```

```
# A tibble: 1 x 1
  avg
<dbl>
1  24.1
```

Median & Mean with dplyr

Use dplyr grammar to find the frequency, mean, and median.

Add lots of summary variables using functions inside `summarise()`.

1. Use `n()` function to create a count variable (no variable in parentheses)
2. Use `mean()` function on a variable
3. Use `median()` function on a variable

```
gss_all |>
  drop_na(agekdbnr) |>
  summarise(
    freq = n(),
    med = median(agekdbnr),
    avg = mean(agekdbnr)
  )
```

```
# A tibble: 1 x 3
  freq med avg
<int> <dbl> <dbl>
1 28867    23 24.1
```

Use dplyr grammar to find the frequency, mean, and median for the variable: `hrs1`.

```
gss_all |>
  drop_na(hrs1) |>
  summarise(
    # ADD TO THE PIPE HERE WITH YOUR CODE HERE
  )
```

group_by() and summarize()

A common pipeline is `group_by()` and then `summarize()` to calculate summary statistics for each group:

Use `group_by()` to organize your data into groups based on one or more variables.

Use `summarize()` to compute statistics like total, mean, or median for each group.

1. Start with the `gss_all` data frame:
2. Remove any observations with missing data for our key variables
3. Do the next steps separately for each group in the variable
4. Creates a new data frame with one row for each combination of grouping variables

```
gss_all |>
  drop_na(sex, agekdbrn) |>
  group_by(sex) |>
  summarise(avg = mean(agekdbrn))
```

```
# A tibble: 2 x 2
  sex      avg
<dbl+lbl> <dbl>
1 1 [male]  25.7
2 2 [female] 23.0
```

Compare the average and median work hours (`hrs`) for U.S. men and women.

```
1 gss_all |>
2   drop_na(sex, hrs1) |>
3   # ADD TO THE PIPE HERE WITH YOUR CODE HERE
```

summarize() + variability measures

```
gss_all |>
  drop_na(agekdbrn, sex) |>
  group_by(as_factor(sex)) |>
  summarise(
    freq = n(),
    min = min(agekdbrn),
    median = median(agekdbrn),
```

```

max = max(agekdbnrn),
mean = round(mean(agekdbnrn), digits = 2),
sd = sd(agekdbnrn)
)

```

```

# A tibble: 2 x 7
  `as_factor(sex)`   freq min      median max      mean    sd
  <fct>             <int> <dbl+lbl> <dbl> <dbl+lbl> <dbl> <dbl>
1 male              11790 9          25 65      25.7  5.85
2 female            17009 9          22 57      23.0  5.30

```

Think Like a Statistician

How did the mean and standard deviation of weekly work hours differ between men and women in 2024, and what do these differences suggest about gender-based variability in labor patterns.

How do we find out?

```
# ADD YOUR CODE HERE
```

Your Data Take

What's your conclusion to our initial research question?